

COMPSCI5079 (M)

Cryptography & Secure Development

Dr. Nguyen Truong

Lecturer (Assistant Professor), SoCS, University of Glasgow

Email: Nguyen.Truong@Glasgow.ac.uk

Office: SAWB 221C, Sir Alwyn Williams Building, 18 Lilybank Gardens



Lecture 5:

Public-key Cryptography





Outline

1. Overview
 - Introduction
 - A Big Picture
2. Diffie Hellman Key Exchange Protocol
3. RSA Public key Cryptography
 - KeyGen
 - Encryption / Decryption
 - Digital Signature
4. Elliptic Curves Cryptography

Lecture 5:

3. RSA Public-key Cryptography





RSA Algorithm

- Invented in 1978 by Ron Rivest, Adi Shamir and Leonard Adleman
- Published as R L Rivest, A Shamir, L Adleman, "On Digital Signatures and Public Key Cryptosystems", Communications of the ACM, vol 21 no 2, pp120-126, Feb 1978.
- Security relies on the difficulty of factoring large composite numbers
- Essentially the same algorithm was discovered in 1973 by Clifford Cocks, who works for the British intelligence



RSA Example

Alice wants to send Bob a message M using RSA

1. Bob generates a key-pair (pk, sk) ; pk is publicly available.
2. Alice encrypts the message M with Bob's public-key pk
3. Bob decrypts the ciphertext C with his private-key sk



RSA Example: Key Generation (KeyGen)

Key generation: how to generate (pk, sk) for Bob

1. Select 2 large prime numbers of about the same size, p and q
2. Compute $n = pq$, and $\Phi(n) = (q-1)(p-1)$
3. Select e , $1 < e < \Phi(n)$, s.t. $\gcd(e, \Phi(n)) = 1$
4. Compute d , $1 < d < \Phi(n)$ s.t. $ed \equiv 1 \pmod{\Phi(n)}$

Then the key pair is formulated as follows:

Public key: $pk = (e, n)$; Private key: $sk = (d, n)$



RSA Example: Encryption and Decryption

Encryption:

1. Given a message M , $0 < M < n$; $M \in \mathbb{Z}_n - \{0\}$
2. Use the public key $pk = (e, n)$
3. Compute ciphertext $C = M^e \bmod n$; $C \in \mathbb{Z}_n - \{0\}$

Decryption:

1. Given a ciphertext C
2. Use the private key $sk = (d, n)$
3. Compute $C^d \bmod n = (M^e \bmod n)^d \bmod n = M^{ed} \bmod n = M$



RSA Example

Key generation

- $p = 11, q = 7, n = 77, \Phi(n) = 60$
- $d = 13, e = 37$ ($ed = 481; ed \bmod 60 = 1$)

Encryption:

- Let $M = 15$. Then $C \equiv M^e \bmod n$

$$C \equiv 15^{37} \pmod{77} = 71$$

Decryption:

- $M \equiv C^d \bmod n$

$$M \equiv 71^{13} \pmod{77} = 15$$



Why does this RSA work?
Any proof?



Reduced Set of Residues

- The RSA algorithm is based on a small amount of number theory.
- The reduced set of residues (RSR) of a number n is the set of all integers which have an inverse mod n .
 - In other words, all the numbers a with:
 $\gcd(a, n) = 1$
E.g., RSR of 10 = $\{1, 3, 7, 9\}$
 - RSR of prime $p = \{1, 2, \dots, p-1\}$



Euler Totient Function

- The *Euler Totient Function* ϕ is the size of this set.
 - $\phi(10) = 4$.
 - $\phi(p) = p-1$, where p is prime.
 - $\phi(pq) = (p-1)(q-1)$, where p, q are prime.
 - $\phi(pqr) = (p-1)(q-1)(r-1)$, where p, q, r are prime.
- This second result can be proved by counting.
 - There are $pq-1$ numbers in the range $1 \dots pq-1$



Euler Totient Function

- We do not want factors of p : $p, 2p, \dots, (q-1)p$
 - Thus, here are $q-1$ of these.

- Similarly, there are $p-1$ factors of q .

- There are no numbers with factors p and q :

$$\begin{aligned}\phi(pq) &= (pq-1) - (q-1) - (p-1) \\ &= pq - p - q + 1 \\ &= (p-1)(q-1)\end{aligned}$$



Fermat's Little Theorem

- If p is prime and $\gcd(a, p) = 1$, then

$$a^{p-1} \bmod p = 1$$

- Proof from Euler's generalization, which follows next.

- Euler's Generalization of Fermat's Theorem

$$a^{\phi(n)} \bmod n = 1, \text{ provided } \gcd(a, n) = 1$$

- Proof beyond the scope of this course.



A Probabilistic Primality Test

- Let a be a prime number.
 - If $a^{n-1} \% n \neq 1$, then n is NOT prime.
 - If $a^{n-1} \% n = 1$, then n may be a prime, or the result may be 1 by accident.
- Choose a series of prime numbers a , and calculate $a^{n-1} \% n$.
 - If the answer is 1 in all cases, then we can assume that n is prime with a small probability of error.
- Carmichael's numbers are not prime but always pass this test.
 - There are not many of them.



Exponentiation Ciphers

- The RSA public key system is a form of exponentiation cipher, so we will consider exponentiation ciphers first.
- The Plaintext Input
 - Let us choose an integer n and split the plaintext into a series of blocks P_i with
$$0 \leq P_i < n.$$
 - I will drop the i suffix from now on and just consider what happens to one block.



Exponentiation Ciphers

➤ Encryption

- Choose an integer e and calculate the cipher text $C = P^e \% n$

➤ Decryption

- Choose an integer d and recover the message by $P = C^d \% n$

➤ d and e must be inverses, so that

- $de \equiv 1 \pmod{\phi(n)}$
- Hence $de = 1 + \lambda \phi(n)$ for some integer λ .
- We don't need to know what λ is.
- Also, $\gcd(d, \phi(n)) = 1$ for the inverse to exist.
- Therefore, d is the unique inverse of e .



Proof

$C = P^e \pmod n$ (Ciphertext C : encrypt plaintext P)

$P' = C^d \pmod n$ (Plaintext P' = decrypt ciphertext P)

→ We want to show: $P' = P$

$$\begin{aligned} P' &= C^d \pmod n \\ &= P^{ed} \pmod n \\ &= P^{1+\lambda\phi(n)} \pmod n \\ &= P * P^{\lambda\phi(n)} \pmod n \\ &= P * (P^{\phi(n)} \pmod n)^{\lambda} \pmod n \\ &= P, \text{ since } P^{\phi(n)} \pmod n = 1 \text{ and } (1^{\lambda} \pmod n = 1) \end{aligned}$$

By Euler's generalisation of Fermat's theorem, assuming $\gcd(P, n) = 1$.



Rivest Shamir Adleman (RSA) Scheme

- Choose $n = pq$, the product of two primes.
- Then $\phi = (p-1)(q-1)$ which can only be calculated if n is factored.
 - We have to know both p and q to calculate ϕ .
- This is safe provided our enemies cannot factor n into two primes.
- $\gcd(P, n)$ may not be 1 if P has either p or q as a factor.
 - So, the encryption fails because Euler's generalisation does not apply.
- The values of p and q are so large that this is very unlikely and can be ignored.



Difficulty of Factoring

- Factoring is not known to be NP-complete.
 - Primality testing is similar to factoring. A polynomial-time algorithm for primality testing was discovered in 1995.
 - There is no-known polynomial time algorithm for factoring.
- The best general-purpose factoring algorithm is $O(n^{\sqrt{(\log \log n) / \log n}})$
- This would involve about 10^{15} operations for a 100-digit number.



RSA Security

- Security depends on the difficulty of factoring n :
 - Factor $n \Rightarrow \Phi(n) \Rightarrow$ compute d from $(e, \Phi(n))$
- The length of $n=pq$ reflects the strength
 - 700-bit n factored in 2007
 - 768 bit factored in 2009
- 1024 bit for a minimal level of security today
 - Likely to be breakable in the near future
 - Minimal 2048 bits recommended for current usage
- NIST suggests 15360-bit RSA keys are equivalent in strength to 256-bit
- RSA speed is quadratic in key length



Real-world Usage

- Often used to encrypt a symmetric key
 - To encrypt a message M under a public key (n, e) , generate a new AES key K , compute $[RSA(n, e, K), AES(K, M)]$
- Original RSA is not secure enough and often needs padding, e.g., Optimal Asymmetric Encryption Padding (OAEP)
 - Roughly, to encrypt M , choose random r , and encode M as:
$$M' = [X = M \oplus H_1(r), Y = r \oplus H_2(X)]$$
where H_1 and H_2 are cryptographic hash functions, then encrypt it as $(M')^e \bmod n$
 - Note that given $M' = [X, Y]$, $r = Y \oplus H_2(X)$, and
$$M = X \oplus H_1(r)$$



Summary

➤ RSA algorithm

- How to generate the key pair

Public key: $pk = (e, n)$; Private key: $sk = (d, n)$

- How to encrypt using the public key

Compute ciphertext $C = M^e \bmod n$

- How to decrypt using the private key

Compute $M' = C^d \bmod n$
 $= (M^e \bmod n)^d \bmod n$
 $= M^{ed} \bmod n = M$



Quizzes

Describe the RSA public key encryption system and show how it works when prime numbers 5 and 7 are used to construct the modulus $n = 35$.

- Choose an encryption parameter e and calculate the corresponding decryption parameter d .
- Use these values to encrypt the plaintext value 17 and decrypt the resulting ciphertext to recover the plaintext.



Digital Signature





Recall: Key pairs

- Instead of sharing a key between pairs of parties ...
- ...every party has a pair of keys
 - public key: published for the world to see
 - private key: kept secret and never shared



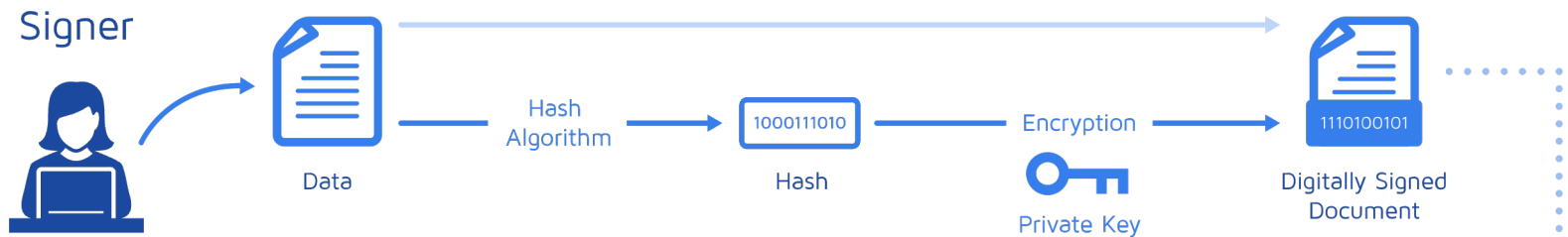


Key-pair Terminology

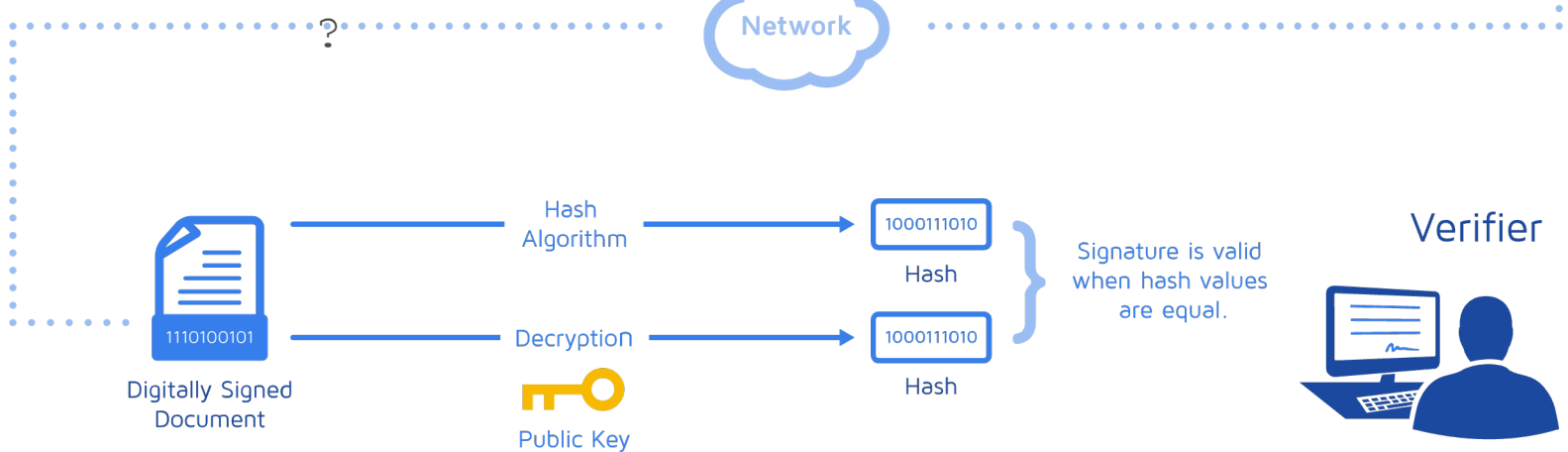
	Encryption	Digital Signatures
Public key	Encryption key	Verification key
Private key	Decryption key	Signing key



How does Digital Signatures work?



How does Digital Signatures work





Digital Signature Scheme

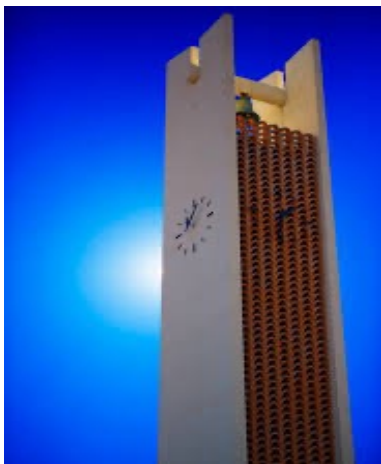
- Digital Signature: a data string which associates a message with some originating entity. One party generates a signature, and many parties can verify.
- Digital Signature Scheme:
 - a signing algorithm: takes a message and a (private) signing key, outputs a signature
 - a verification algorithm: takes a (public) key verification key, a message, and a signature
- Provides:
 - Authentication, Data integrity, Non-Repudiation



Digital Signature Scheme

A digital signature scheme is a triple $(\text{Gen}, \text{Sign}, \text{Ver})$:

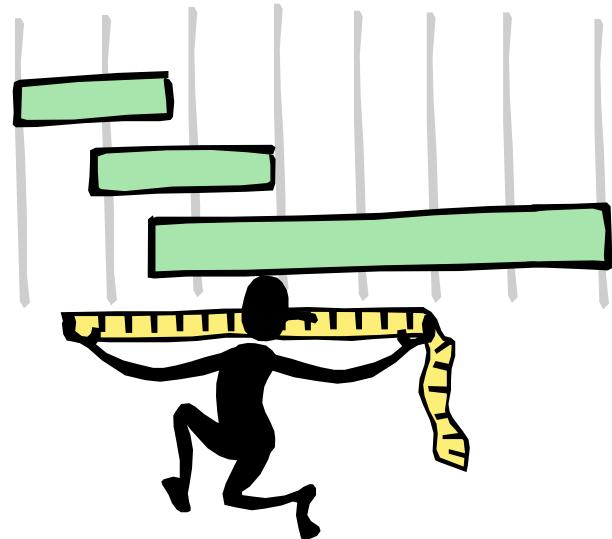
- $\text{Gen}(\text{len})$: generate a key pair (pk, sk) of length len
- $\text{Sign}(m; \text{sk})$: sign message m with private key (i.e., signing key) sk , producing signature s as output
- $\text{Ver}(m, s; \text{pk})$: verify signature s on message m with public key (i.e., verification key) pk





Digital Signature and Message Digest

- Very often digital signatures are used with hash functions, a hash of a message is signed, instead of the message.
- The hash function must be:
 - Pre-image resistant
 - Weak collision resistant
 - Strong collision resistant





RSA Digital Signature

Key generation (as in RSA encryption):

1. Select 2 large prime numbers of about the same size: p and q
2. Compute $n = pq$, and $\Phi = (q - 1)(p - 1)$
3. Select a random integer e , $1 < e < \Phi$ s.t. $\gcd(e, \Phi) = 1$
4. Compute d , $1 < d < \Phi$ s.t. $ed \equiv 1 \pmod{\Phi}$

Public key: (e, n) used for verification

Secret key: (d, n) used for generation



RSA Digital Signature

Signing message M

- Verify $0 < M < n$
- Compute $S = M^d \bmod n$

Verifying signature S

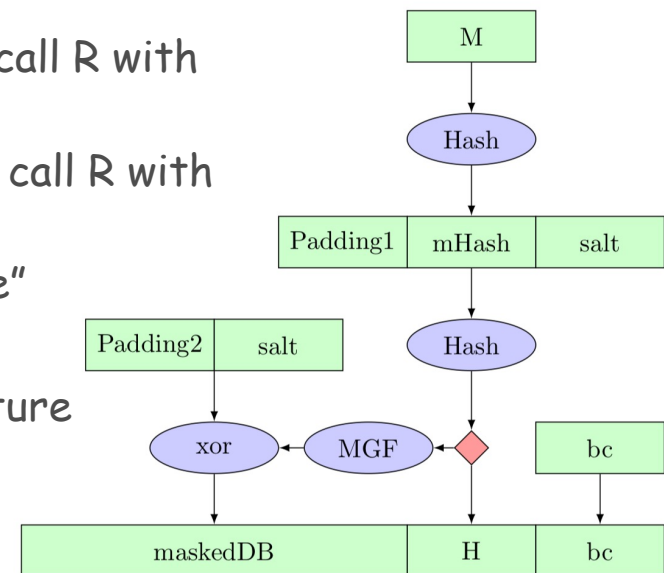
- Use public key (e, n)
- Compute $S^e \bmod n = (M^d \bmod n)^e \bmod n = M$

Note: in practice, a hash of the message is signed and not the message itself.



RSA Digital Signature

- Core ideas are the same as RSA encryption
- Common mistake: "RSA sign = encrypt with private key"
- In practice, outside of textbooks:
 - There's a core RSA function R that works with either pk or sk
 - RSA encrypt = do some prep work on m then call R with pk
 - RSA sign = do different prep work on m then call R with sk
 - Prep work (padding): "original RSA is insecure"
 - For encryption: OAEP
 - For signatures: PSS (probabilistic signature scheme)
 - Also, need to handle long messages...



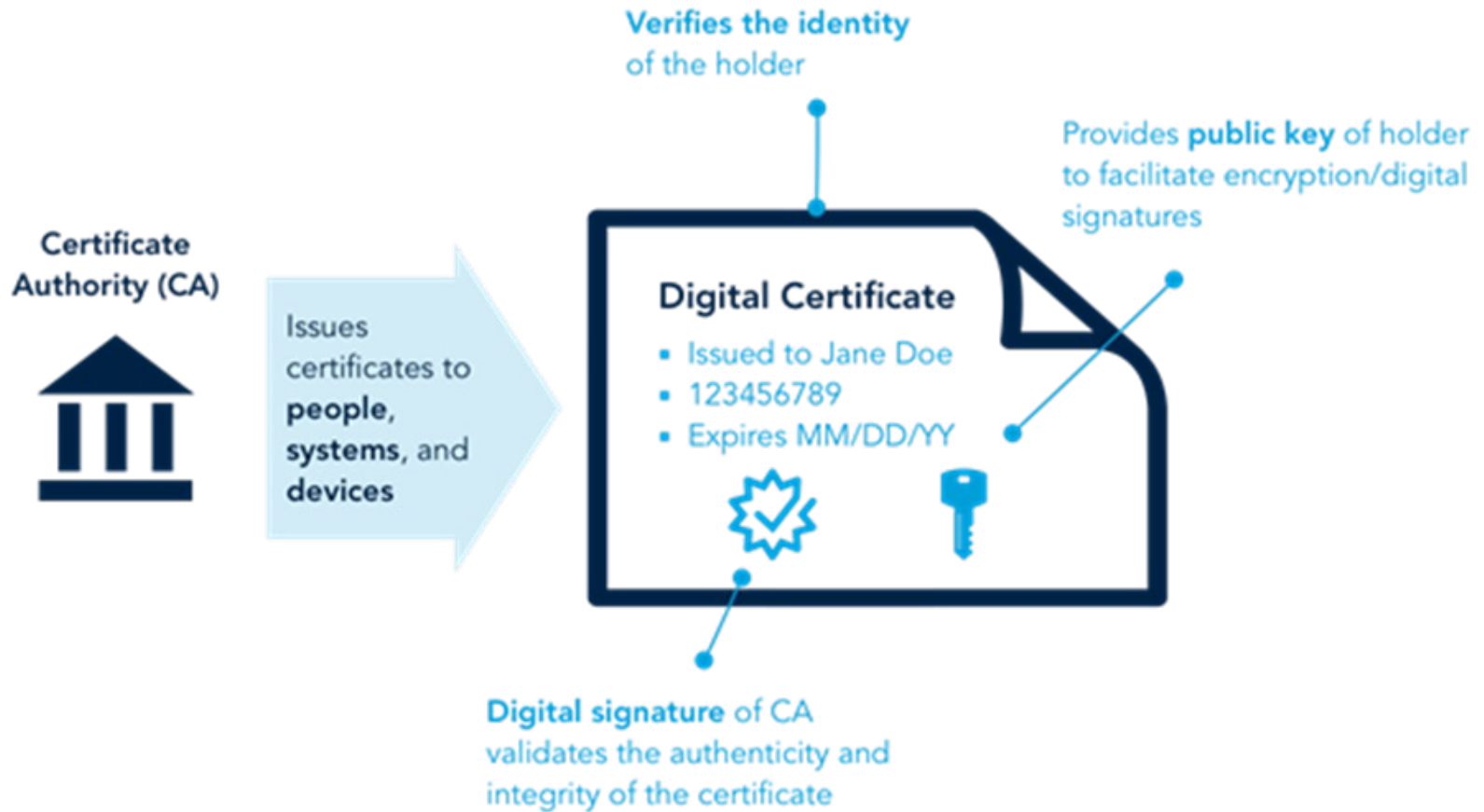


RSA Digital Signature

- DSA: Digital Signature Algorithm [Kravitz 1991]
 - Standardized by NIST and made available royalty-free in 1991/1993
 - Used for decades without any serious attacks
 - Closely related to Elgamal encryption



Digital Signatures and Digital Certificates





Summary

► Digital Signature based on RSA:

A triple $(\text{Gen}, \text{Sign}, \text{Ver})$

- $\text{Gen}(\text{len})$: generate a key-pair (pk, sk)
- $\text{Sign}(m; \text{sk})$: sign a message m with private key sk , output signature s
- $\text{Ver}(m, s; \text{pk})$: verify signature s with public key pk



Tutorials / Practice

OpenSSL is an amazing tool that does a variety of tasks, including encrypting files.

1. Using OpenSSL to generate key pair: Alice generates her keys

```
$openssl genrsa -aes128 -out alice_private.pem  
1024
```

→ Generating the private key

```
$openssl rsa -in alice_private.pem -pubout >  
alice_public.pem
```

→ Extracting the associated public key

To view the keys: `$openssl rsa -in alice_private.pem -noout -text`



Tutorials / Practice

OpenSSL is an amazing tool that does a variety of tasks, including encrypting files.

2. Bob sends his public key to Alice

```
$ scp bob_public.pem alice@alice-machine-or-ip:/path/
```

3. Alice encrypts a file using Bob public key

```
$ openssl rsautl -encrypt -inkey bob_public.pem  
-pubin -in top_secret.txt -out top_secret.enc
```

(Note: some newer versions of openssl use pkeyutl instead of rsautl)



Tutorials / Practice

OpenSSL is an amazing tool that does a variety of tasks, including encrypting files.

4. Alice sends the ciphertext (i.e., encrypted file) to Bob

```
$ scp top_secret.enc bob@bob-machine-or-ip:/path/
```

5. Bob decrypts the ciphertext using his private key

```
$ openssl rsautl -decrypt -inkey  
bob_private.pem -in top_secret.enc >  
top_secret.txt
```

6. Try many more examples with different keys to see what happens if the key is incorrect